김수환 포트폴리오

프로젝트 화면과 구조도를 중심으로 구현 방식과 설계 기준을 설명합니다.

Bitria 암호화폐 브로커리지 플랫폼

2025.11 – 현재 · 운영 중　웹·iOS·Android·데스크톱 프론트엔드 전담(앱 스토어 출시)·백엔드 API 개발 · 2인 팀

외부 거래소 유동성으로 현물·선물·마진 거래와 계정·자산 관리를 제공하는 암호화폐 브로커리지 플랫폼입니다.

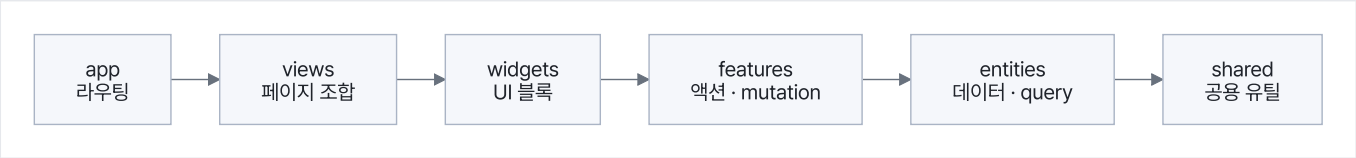
2인 팀에서 프론트엔드 전체를 초기 구축부터 개발하고, 백엔드 API 개발에도 참여했습니다.
아래 자료는 화면·도메인 분리, 실시간 데이터 전달, 멀티 플랫폼 구성을 설명합니다.

구현 구조와 설계

FSD 아키텍처와 화면 분기 기준

화면과 도메인이 늘어나도 의존성이 뒤섞이지 않도록 app → views → widgets → features → entities → shared 순의 단방향 규칙을 정했습니다.
조회(query)는 entity에, 변경(mutation)은 feature에 두어 각 레이어의 책임을 분리했습니다.

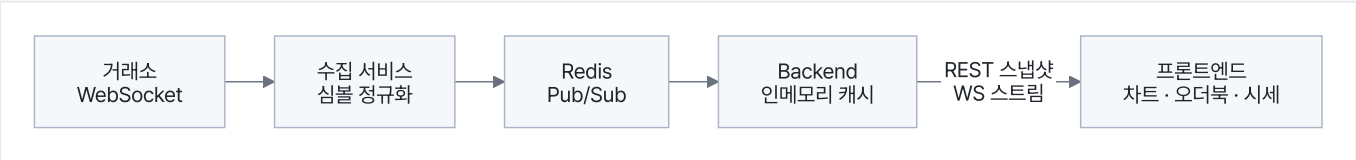
웹과 앱의 라우트가 다르면 폴더를 분리했습니다.
화면 크기 차이가 작으면 prop으로 분기하고, 마크업 차이가 크면 디바이스별 파일을 두도록 기준을 정했습니다.



FSD 레이어 간 의존 방향과 각 레이어의 책임

실시간 데이터 전달 구조

거래소 WebSocket 데이터는 수집 서비스의 심볼 정규화와 Redis Pub/Sub을 거쳐 백엔드에 전달됩니다.
프론트엔드는 REST 스냅샷과 WebSocket 스트림으로 차트·호가·시세를 표시합니다.

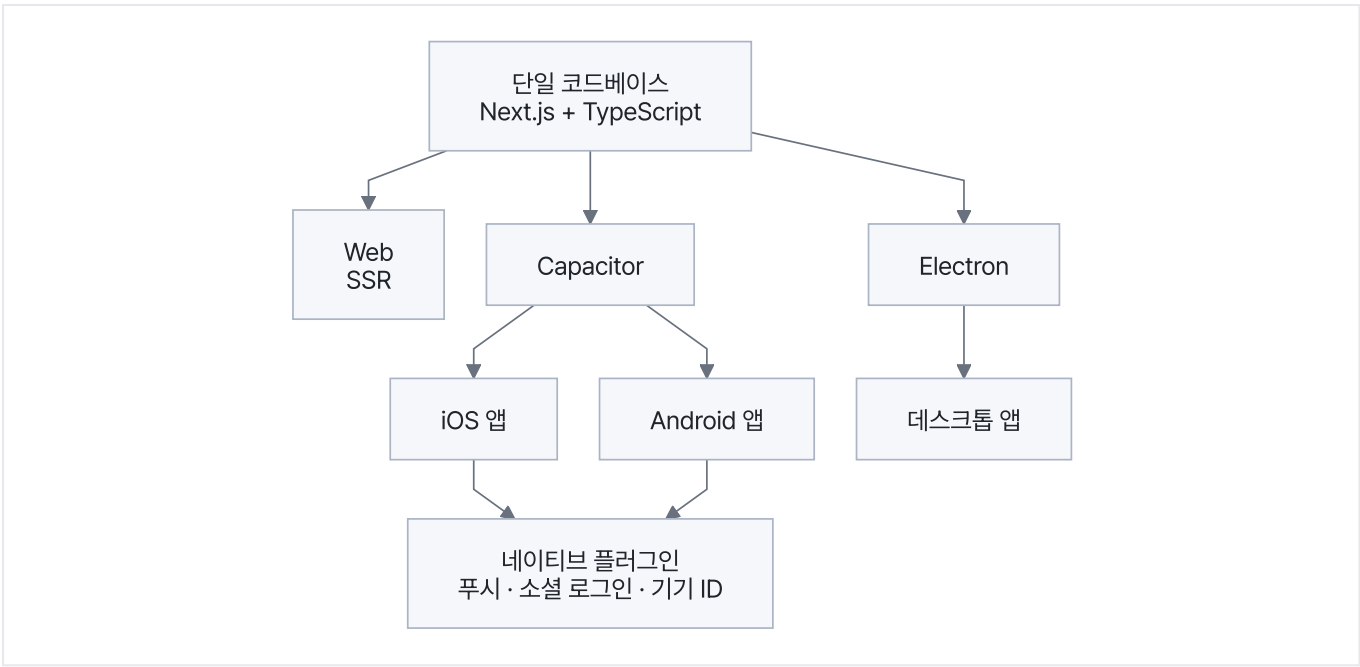


실시간 데이터 전달 경로

단일 코드베이스의 멀티 플랫폼 대응

2인 팀에서 웹·iOS·Android·데스크톱을 빠르게 함께 내기 위해, 각 플랫폼을 따로 만들지 않고 기존 Next.js 코드베이스를 그대로 쓰는 Capacitor와 Electron을 선택했습니다.
Next.js·TypeScript 코드베이스를 웹, Capacitor 기반 iOS·Android 앱, Electron 데스크톱 앱으로 빌드했습니다.
모바일 앱을 스토어에 출시하고 푸시(APNs·FCM)·소셜 로그인·기기 식별 기능을 연결했습니다.

앱 강제 업데이트 정책을 연동하고, Electron 앱의 API 요청을 식별하는 헤더를 추가했습니다.



웹·모바일·데스크톱 빌드와 네이티브 기능 연결 구조

문제 해결과 개선

비활성 화면의 불필요한 리렌더 개선

차트 사전 로딩을 위해 숨겨둔 화면이 시세 갱신마다 렌더링되며 전체 렌더 시간의 64%를 차지하는 문제를 발견했습니다.
실시간 데이터 구독을 화면 최상위에서 실제 값을 쓰는 컴포넌트로 옮기고, 비활성 화면의 구독을 멈췄습니다.

> 렌더 작업 5,192회 → 1,534회 (70% 감소)

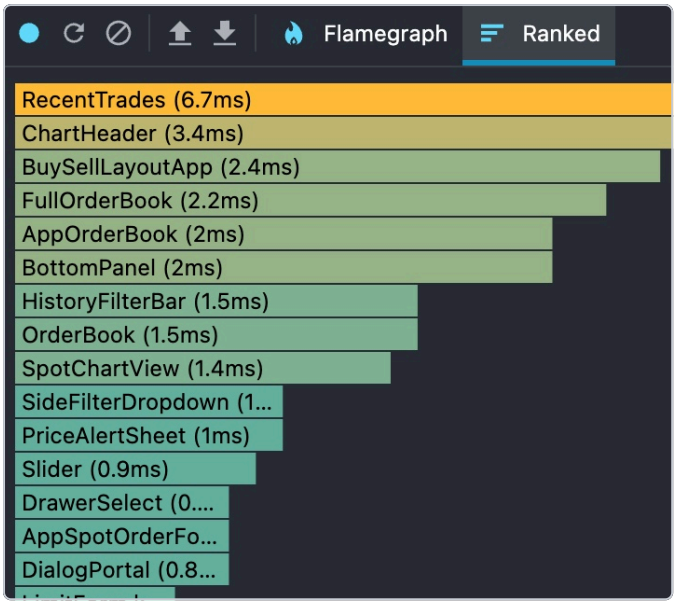
측정 방법: 공개 시세 메시지 73개를 녹화해 개선 전·후 커밋에 같은 12초 동안 재생하고, React DevTools 훅(onCommitFiberRoot)으로 커밋마다 실제로 렌더된 컴포넌트를 집계했습니다. 개발 빌드, 모바일 앱 화면(390×844), 회차마다 새 브라우저로 전후 각 5회 실행한 중앙값입니다.

```
══ 리렌더 측정 요약 (5회 중앙값) ══
  입력: 녹화한 시세 이벤트 73개를 12초 동안 동일하게 재생 (fixture sha256 7d33b33cdb8e…)
  계측: React DevTools 훅(onCommitFiberRoot)으로 커밋마다 실제 렌더된 컴포넌트(PerformedWork) 집계

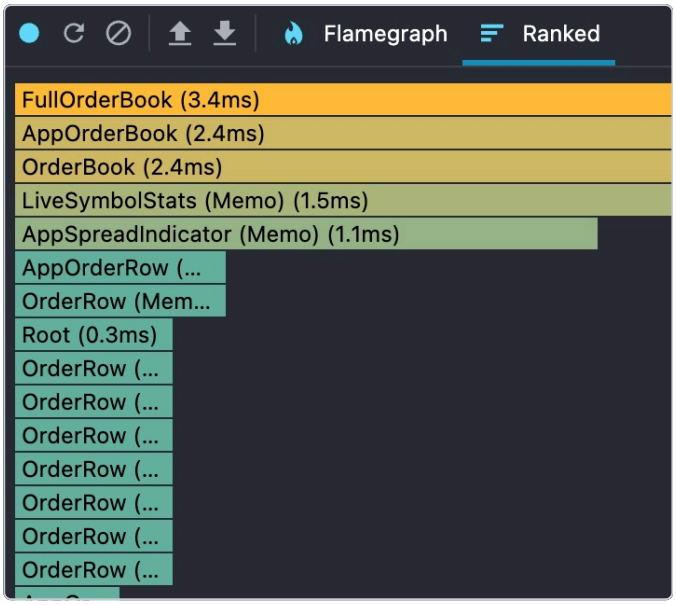
  지표                          개선 전          개선 후    변화
  렌더된 컴포넌트 합계          5,192회   →      1,534회    -70.5%
  React 커밋 수                   93회   →         75회    -19.4%
  React 렌더 시간               661.0ms   →      394.7ms    -40.3%

  컴포넌트별 렌더 횟수 (중앙값)
  BuySellLayoutApp                8회   →          0회    -100.0%
  SpotChartView                  14회   →          0회    -100.0%
  ChartHeader                    14회   →          0회    -100.0%
  AppOrderForm                    8회   →          0회    -100.0%
  BottomPanel                     8회   →          0회    -100.0%
  AppOrderBook                   34회   →         28회    -17.6%
  RecentTrades                   32회   →         18회    -43.8%
```

측정 스크립트 실행 결과 · 렌더된 컴포넌트 합계와 컴포넌트별 렌더 횟수 (5회 중앙값)



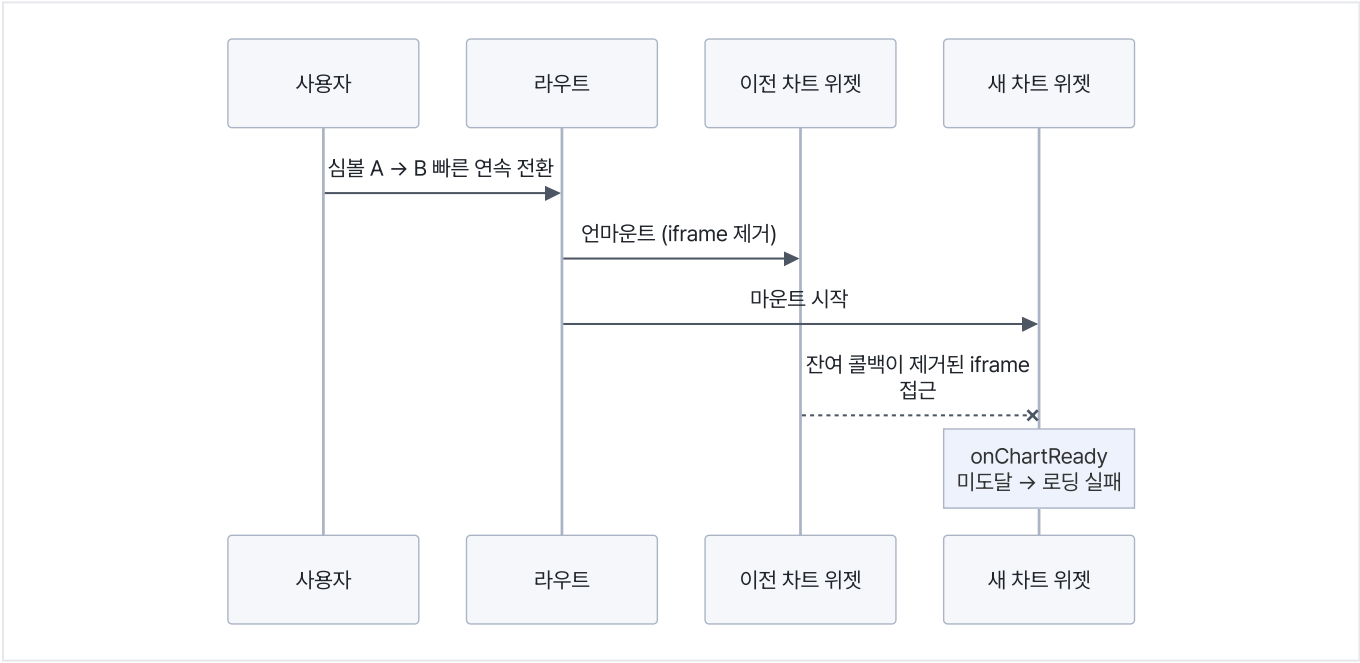
개선 전 · 시세 갱신 한 번에 레이아웃·주문 폼까지 리렌더



개선 후 · 같은 갱신에 값 표시 컴포넌트만 리렌더

TradingView 차트 로딩 안정화

운영 로그로 실패를 추적해 캐시 버그와 심볼 전환 시 경쟁 상태를 원인으로 특정했습니다.
버그와 무관한 로딩 방식으로 전환하고 위젯을 유지한 채 심볼만 교체하도록 바꿨으며, 실패해도 재시도로 복구되도록 했습니다.



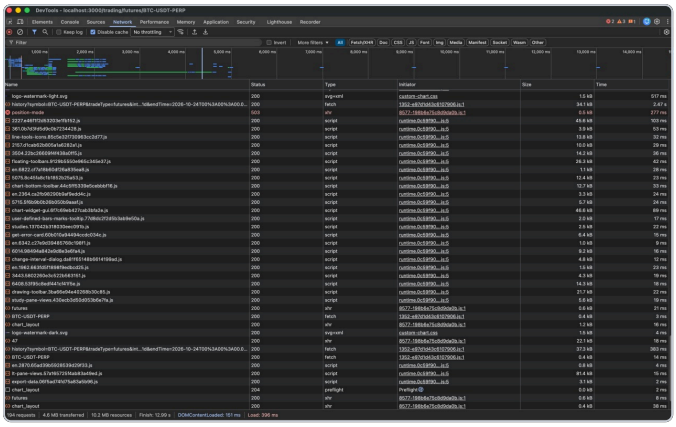
수정 전 심볼 전환 경쟁 상태: 이전 위젯에 남아 있던 콜백이 제거된 iframe에 접근해 새 위젯이 준비 완료에 도달하지 못하는 흐름

배럴 import의 불필요한 모듈 유입 개선

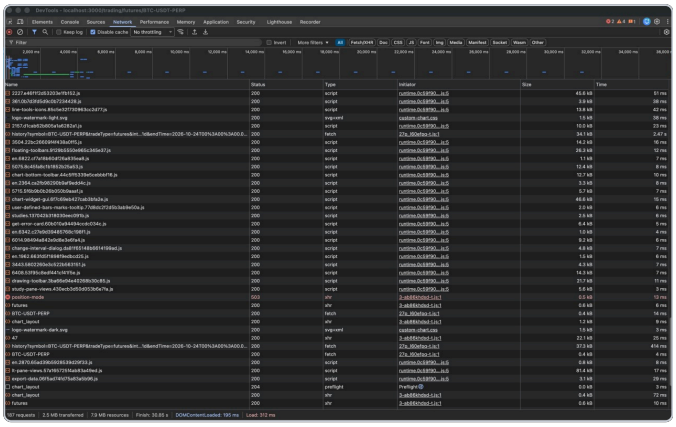
배럴 import 136곳을 직접 참조로 바꿔 모바일 홈 TBT 30% 감소(417 → 292.5ms)

빌드·번들 최적화

Turbopack 전환과 번들·폰트 최적화로 빌드 시간 56%, 홈 JS 27%, 초기 폰트 전송 95% 감소



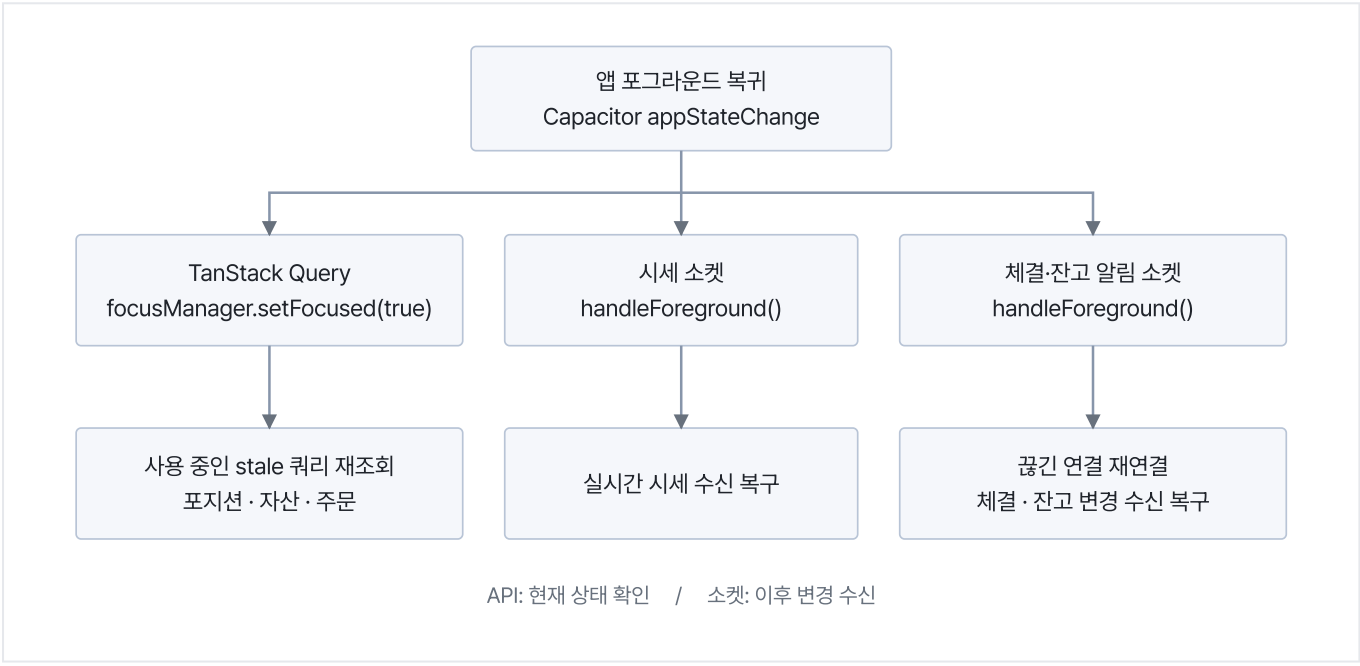
개선 전: 거래 화면 콜드 로드 4.6MB 전송 · 194 요청



개선 후: 같은 조건에서 2.5MB 전송 · 187 요청

앱 복귀 후 포지션·자산 정보 불일치 해결

WKWebView의 복귀 이벤트 누락을 찾아 앱 상태 이벤트로 재조회 연결, 복귀 즉시 최신 포지션·자산 표시

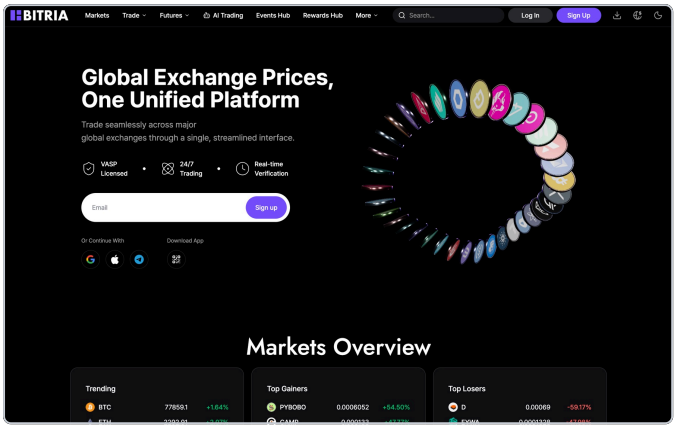


앱 복귀 이벤트에서 현재 상태 재조회와 실시간 연결 복구를 함께 실행하는 흐름

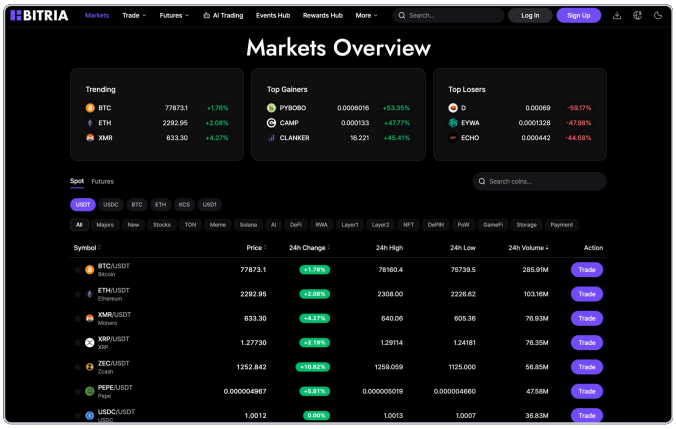
비영어 기기에서 앱 대신 Safari가 열리던 문제 해결

Capacitor·next-intl 경로 판정 충돌을 찾아 rewrite로 해결, 비영어 5개 언어 기기에서 앱 정상 실행

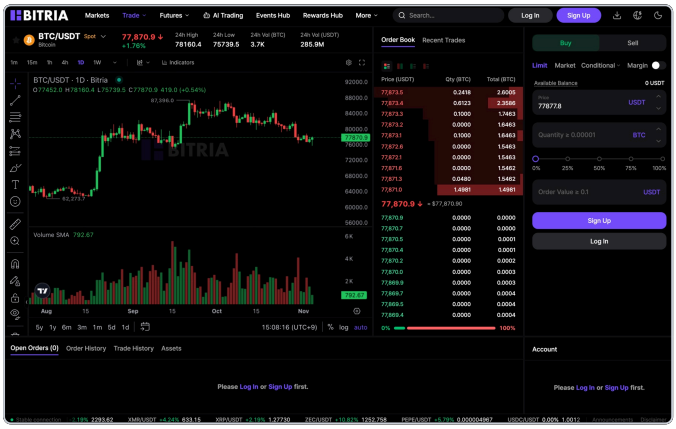
프로젝트 화면



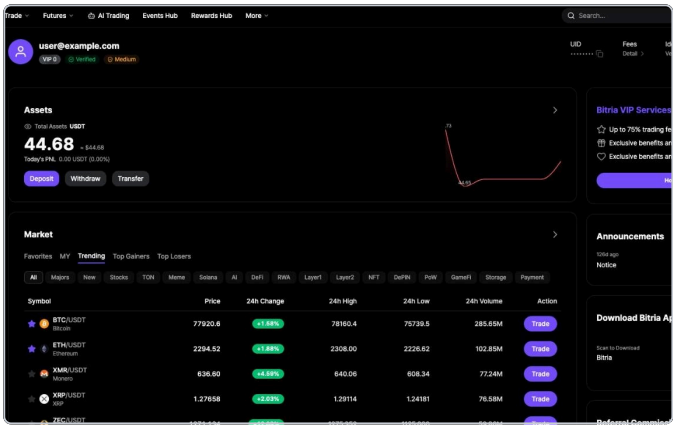
웹 홈 화면



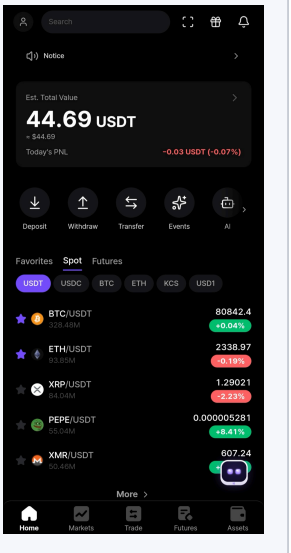
웹 마켓 화면



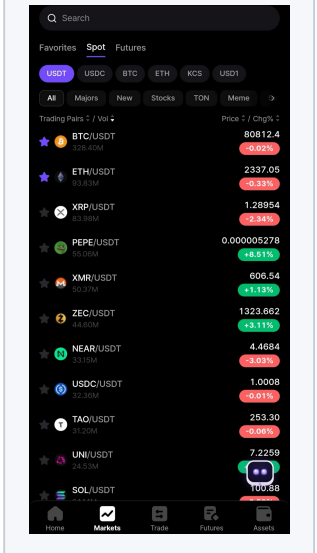
웹 거래 화면



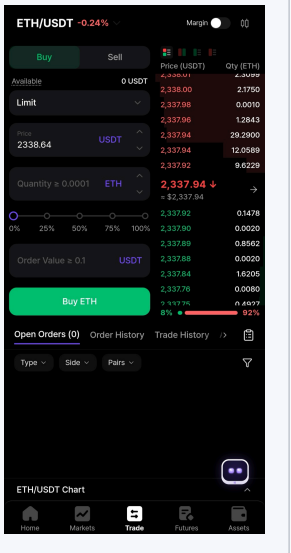
웹 마이페이지 화면 (이메일·UID 가림)



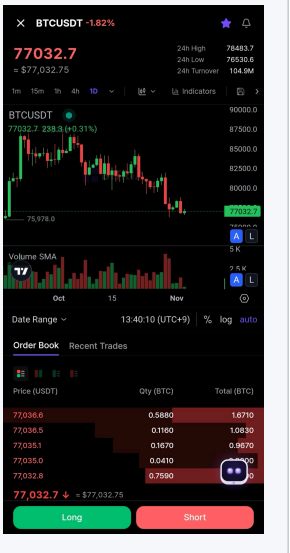
앱 홈 화면



앱 마켓 화면



앱 현물 거래 화면



앱 선물 차트 화면

기술 스택

프론트엔드	TypeScript · Next.js · Tailwind CSS · next-intl · Zustand · TanStack Query · Capacitor · Electron
백엔드	Node.js · Express · Prisma · MariaDB · TimescaleDB · Redis · RabbitMQ · Socket.IO · AWS(EC2) · Docker · Kubernetes(K3s)

Gobit 코인 정보·커뮤니티 플랫폼

2024.10 – 2025.12 프론트엔드·백엔드 개발 담당 · 2인 팀

암호화폐 시세·차트·뉴스·커뮤니티와 모의투자를 제공하는 플랫폼입니다.

2인 팀에서 React SPA와 Next.js 웹 클라이언트, Node.js·Express API를 개발했습니다.
조회 데이터와 실시간 시세의 전달 경로, 레거시 서비스 통합 구조를 설명합니다.

구현 구조와 설계

실시간 시세와 일반 조회 데이터의 분리

서버가 Redis에 캐시한 시세를 Socket.IO로 받아 표시하고, 연결이 끊기면 재연결·헬스체크로 복구하도록 개선했습니다.
시세 이외의 조회 데이터는 SWR로 관리해 실시간 스트림과 API 조회의 역할을 구분했습니다.

TradingView 차트를 연동하고 회원 인증·모의투자·랭킹·채팅 기능을 구현했습니다.

레거시 서비스의 통합·배포 구성

다른 개발자가 만든 레거시 서비스는 소스 코드가 유실돼 바이너리로만 서버 5대에 흩어져 있었습니다.
Gobit·모의투자·코인 매매 시그널 서비스를 모노레포로 통합했습니다.
수집기 → Redis → 서버로 데이터 전달 경로를 정리하고, 서비스별 Dockerfile과 Kubernetes 매니페스트를 구성했습니다.

GitHub Actions로 빌드·배포를 자동화하고, 설정값은 Kubernetes Secret에서 불러오도록 구성했습니다.
DB 백업과 캔들 보관 작업은 CronJob으로 분리했습니다.

문제 해결과 개선

SPA 검색 노출 한계로 Next.js 재구축

SPA 사전 렌더링의 한계를 확인하고 Next.js로 재구축해 코인·게시글 상세를 서버 렌더링

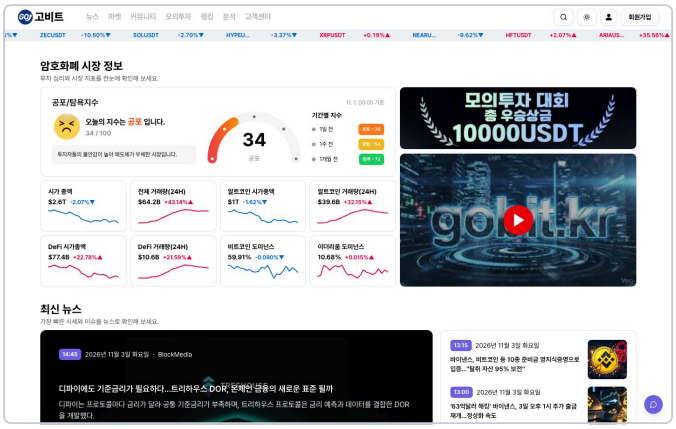
4개 거래소 실시간 시세 연결 구조 정리

호출마다 열리던 WebSocket을 거래소별 단일 연결로 정리하고, 목록은 화면에 보이는 코인만 구독

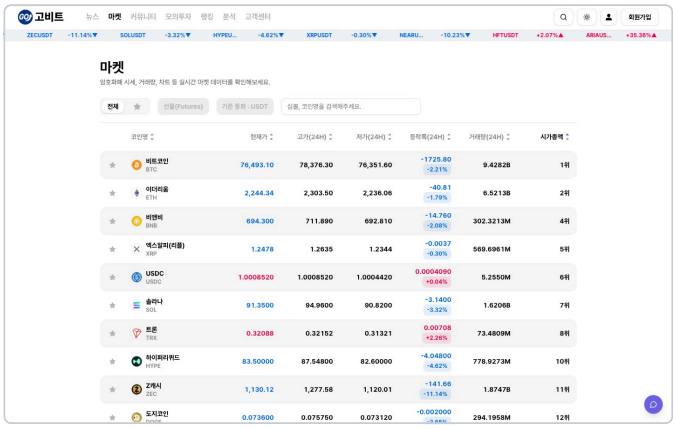
레거시 플랫폼 통합과 Kubernetes 이전

서버 5대에 흩어진 13개 서비스를 K3s로 통합하고, 소스가 유실된 .NET 수집기를 Node.js로 재작성

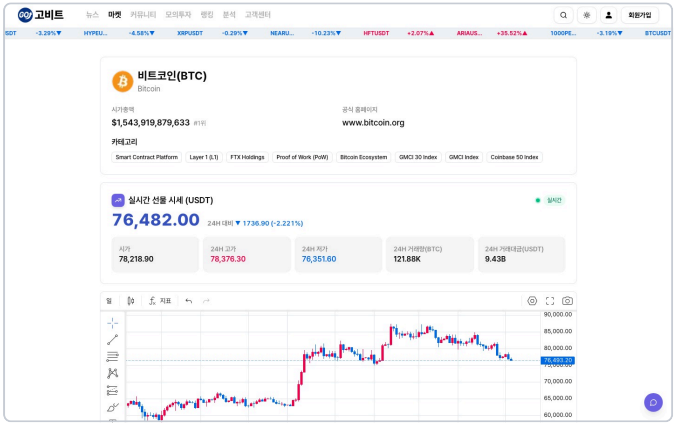
프로젝트 화면



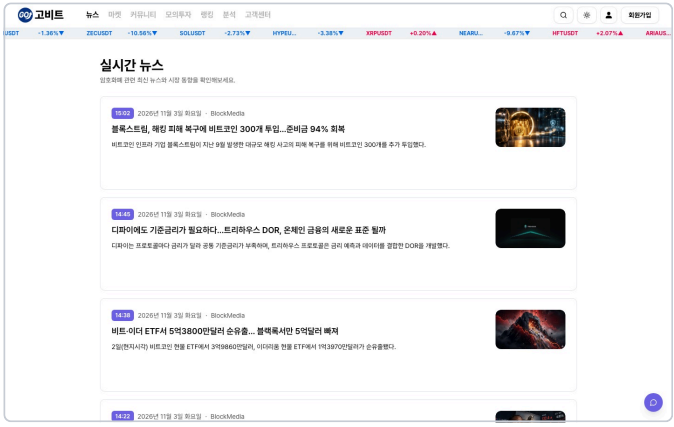
홈 화면



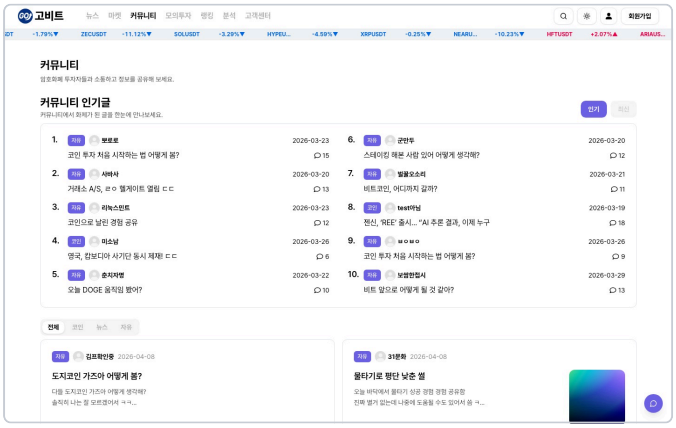
마켓 화면



코인 상세 화면



뉴스 화면



커뮤니티 화면

기술 스택

프론트엔드	TypeScript · Next.js · Tailwind CSS
백엔드	Node.js · Express · MariaDB · Redis · Socket.IO

두잇 홈페이지 리뉴얼 회사 홈페이지 React 마이그레이션

2024.05 – 2024.07 프론트엔드 개발 담당

사내 React 도입에 맞춰 기존 jQuery 기반 회사 홈페이지를 React로 전환하고 새로 단장한 프로젝트입니다.

메인·회사소개·서비스·인증·파트너 페이지를 담당했습니다.
공통 UI 적용과 한국어·영어, 반응형 화면의 구성 방식을 설명합니다.

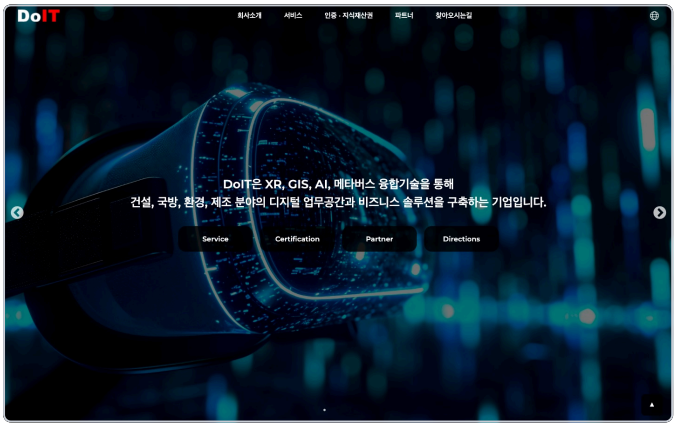
구현 구조와 설계

공통 UI와 페이지별 구성

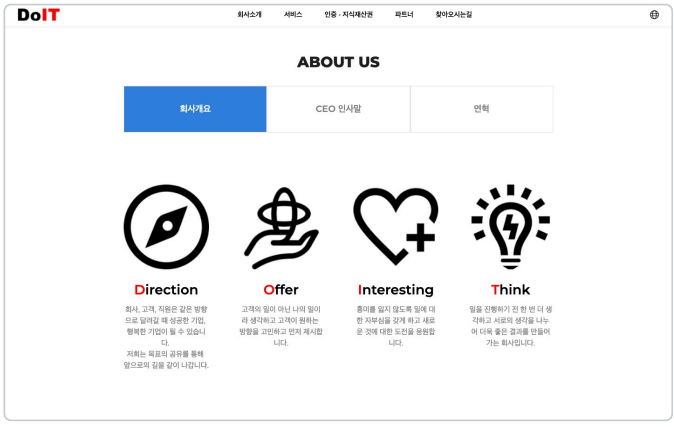
색상·타이포·레이아웃 등의 공통 요소에 Doit UI Kit을 적용하고, 페이지별 콘텐츠를 React 컴포넌트로 구성했습니다.

번역 리소스는 i18next의 페이지별 네임스페이스로 분리했습니다.
CSS와 JavaScript에서 같은 브레이크포인트를 사용하고, 모바일에서 불필요한 영역은 렌더링하지 않도록 구성했습니다.

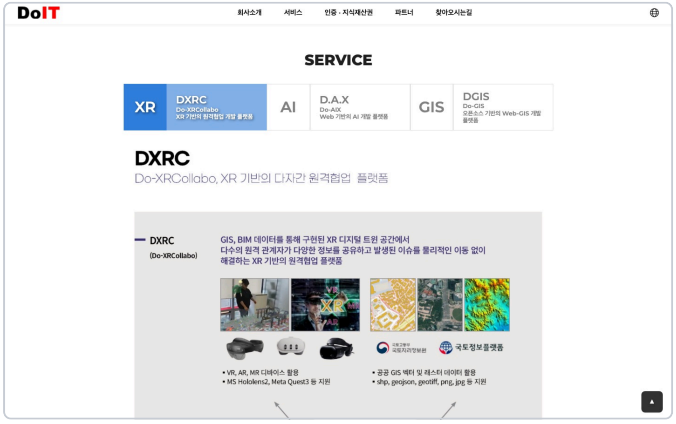
프로젝트 화면



메인 화면



회사소개 화면



서비스 화면



인증 화면

기술 스택

프론트엔드 TypeScript · React · styled-components · i18next

Doit UI Kit 사내 공통 UI 라이브러리

2024.02 – 2024.07 프론트엔드 개발 담당

D.A.X와 회사 홈페이지에 적용한 사내 공통 React UI 라이브러리입니다.

컴포넌트 24종과 패키징·npm 배포·사용 문서를 담당했습니다.
아래 Storybook 화면에서 컴포넌트별 Props와 상태별 UI를 확인할 수 있습니다.

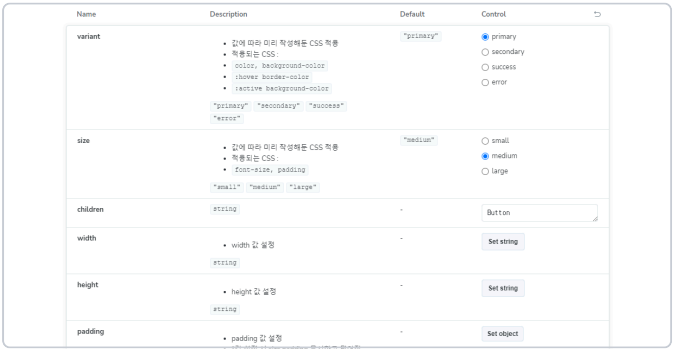
구현 구조와 설계

컴포넌트 조합과 라이브러리 배포

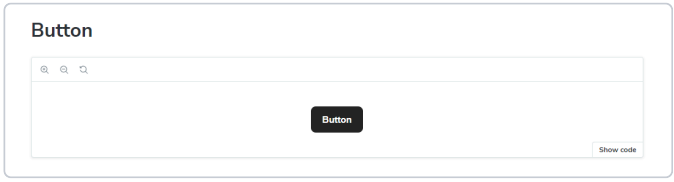
Button·Input 같은 Atom을 조합해 FileInput 등의 Molecule을 구성했습니다.
컴포넌트의 Props와 상태별 UI를 Storybook에 문서화하고, 사용하는 프로젝트에서 같은 기준으로 확인하도록 구성했습니다.
Playground에서는 값을 직접 바꿔 사용 예를 확인할 수 있습니다.

vite-plugin-dts로 타입 선언을 함께 배포하고 React·React DOM은 peer dependency로 분리했습니다.
D.A.X와 회사 홈페이지에 적용하면서 발견한 요구사항을 라이브러리에 반영했습니다.

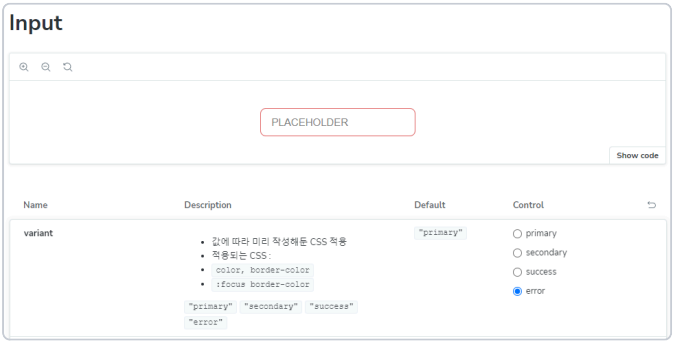
프로젝트 화면



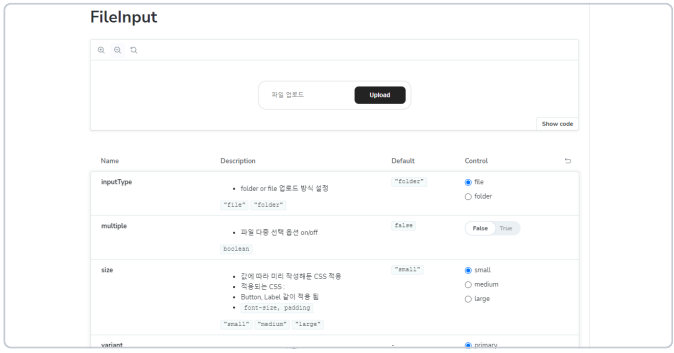
Doit UI Kit 컴포넌트의 Storybook 문서와 Props 설정 화면



Button 컴포넌트의 Storybook 예시



Input 컴포넌트의 상태·Props 문서



Input·Button·Label을 조합한 FileInput 컴포넌트

기술 스택

프론트엔드 TypeScript · React · styled-components · Storybook

D.A.X AI 모델 학습·운영 플랫폼

2024.01 – 2024.06　프론트엔드 개발 담당 · 백엔드 1명과 협업

데이터 관리·라벨링·모델 학습·추론을 지원하는 AI 플랫폼입니다.

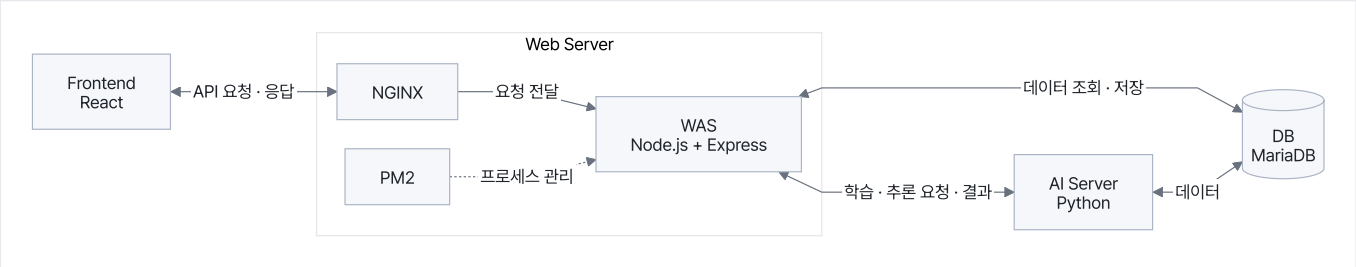
백엔드 개발자 1명과 협업해 프론트엔드를 담당했습니다.
서비스 구성도와 라벨링 화면을 중심으로 API 연동 범위와 좌표 변환 방식을 설명합니다.

구현 구조와 설계

서비스 구성과 프론트엔드 담당 범위

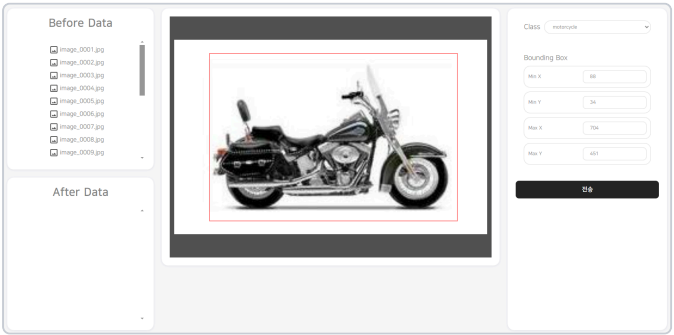
React 프론트엔드는 웹 서버 API를 통해 프로젝트·라벨·모델 데이터를 조회하고 변경합니다.
백엔드와 AI 서버가 연결된 전체 구성에서 프론트엔드 화면과 API 연동을 담당했습니다.

API 호출을 auth·project·class·datamanage·aiModel·operation 등 도메인별 서비스 모듈로 나눴습니다.
축소된 화면에서 그린 좌표는 원본 이미지 좌표로 변환해 저장하고, 다시 표시할 때 역변환했습니다.
양방향 변환은 useCoordinateTransform 훅으로 분리해 그리기·저장·재표시에 같은 규칙을 적용했습니다.

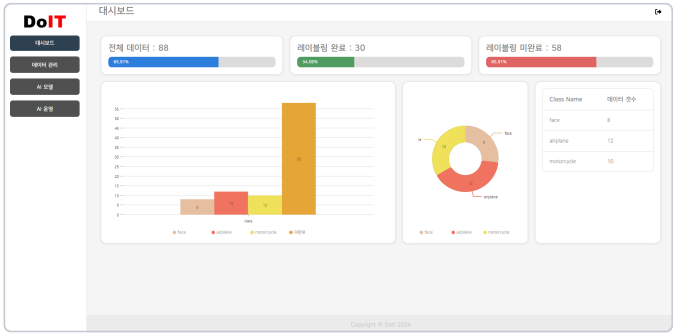


D.A.X 전체 서비스 구성: 담당 범위는 React 프론트엔드와 API 연동

프로젝트 화면



D.A.X에서 이미지 위에 바운딩 박스를 그려 라벨링하는 화면



D.A.X의 프로젝트 데이터와 학습 현황 대시보드

기술 스택

프론트엔드　TypeScript · React · styled-components · TanStack Query · Zustand

XRGIS 백오피스 XR·GIS 원격 협업 시스템 백오피스

2023.08 – 2023.12 프론트엔드·백엔드 개발 담당

XR·GIS 원격 협업 서비스의 사용자·팀·파일 관리 백오피스입니다.

React 화면과 Express API를 담당했습니다.
아래 화면은 사용자·팀 관리와 팀별 파일 접근, 대용량 파일 처리 기능을 보여줍니다.

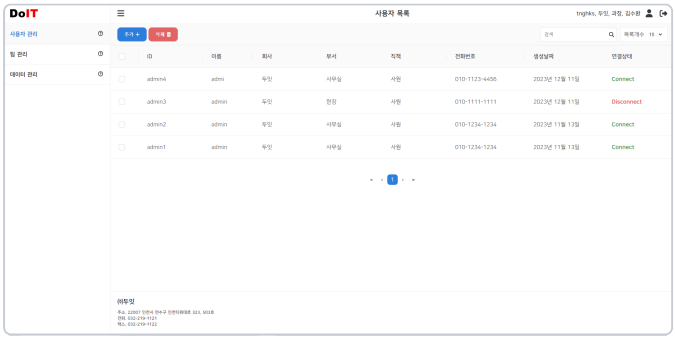
구현 구조와 설계

팀별 접근 권한과 대용량 파일 처리

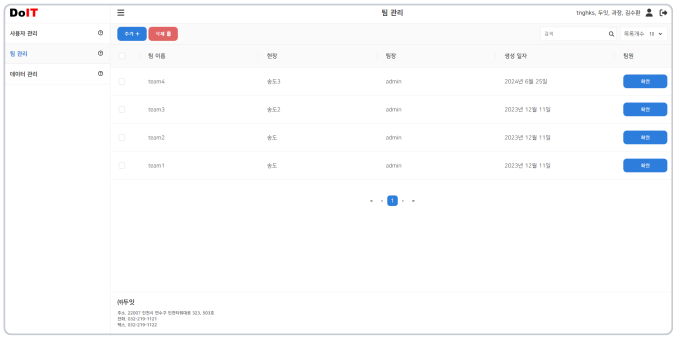
파일 관리 API에서 사용자의 팀 소속에 따라 다운로드 권한을 확인합니다.
같은 관리자 계정을 사용하는 사내 서비스와는 MariaDB 기반 세션 저장소를 통해 인증 체계를 맞췄습니다.

OBJ·영상 파일은 MongoDB GridFS로 청크를 나눠 저장하고, 선택한 파일은 archiver로 ZIP을 생성해 다운로드하도록 구현했습니다.

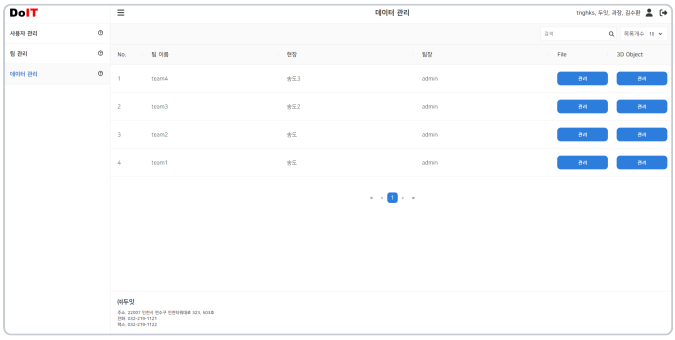
프로젝트 화면



XRGIS 사용자 관리 화면



XRGIS 팀 관리 화면



XRGIS 팀별 데이터·파일 관리 화면: 대용량 파일 관리와 ZIP 다운로드 기능

기술 스택

프론트엔드	TypeScript · React · styled-components
백엔드	Node.js · Express · MongoDB(GridFS) · MariaDB